



**Tecnológico
de Monterrey**

Reflexión Evidencia 1

Pamela Hernández Camacho A01771362

21 de septiembre de 2026

1. Importancia de ordenar y buscar en este tipo de logs

La bitácora es el registro de lo que pasó en un sistema y casi siempre se consulta por ventanas de tiempo. Es decir, por ejemplo, qué ocurrió entre dos horas del día de un incidente.

Cuando varios servidores escriben en el mismo archivo con relojes ligeramente desfasados, el orden de llegada no es el cronológico (justo lo que simula log607-2.txt) y reconstruir una secuencia de eventos fuera de orden puede llevar a conclusiones equivocadas. Si los datos se ordenan una sola vez, cada consulta de rango se puede resolver con búsqueda binaria en máximo 13 pasos en lugar de recorrer los 6,818 registros y con millones de líneas esa diferencia es enorme. Además, los eventos con la misma fecha y hora no deben perderse ni contarse dos veces en una investigación fue por eso que definí el rango como un intervalo cerrado y lo probé con las fechas repetidas del archivo.

2. Comparación de los algoritmos

Algoritmo	Fortaleza	Debilidad	Caso de uso
Burbuja (con bandera)	Simple; se detiene cuando una pasada ya no intercambia nada	Muy lenta con datos desordenados (unos 23 millones de comparaciones)	Datos casi ordenados y pequeños; enseñanza
Selección	Pocos intercambios (a lo más uno por posición)	Siempre revisa todo lo que falta; no aprovecha el orden	Cuando mover datos es muy costoso
Swap Sort	Muy fácil de escribir	Enormes cantidades de intercambios en desorden; siempre las mismas comparaciones	Didáctico
Inserción	Muy rápida con datos casi ordenados; no usa memoria extra	Cuadrática con datos desordenados	Logs casi ordenados; listas pequeñas
Shell	Los saltos grandes acercan registros lejanos; mucho mejor que Inserción en desorden	Su costo depende de la secuencia de saltos	Tamaños medianos sin memoria extra
Merge	$n \log n$ en cualquier caso	Necesita memoria extra del tamaño de los datos	Orden desconocido; datos grandes
Quick (mediana de 3)	Muy rápido en la práctica	Peor caso cuadrático si el pivote es malo	Uso general, con buena elección de pivote

Por qué Inserción (y Burbuja con bandera) se comportan distinto con datos casi ordenados.

Su trabajo es proporcional a n más el número de inversiones, es decir, de pares de registros que están en orden contrario. En log607-1.txt hay 11,593,691 inversiones (el conteo de intercambios de mi Swap Sort); en log607-2.txt solo 17,547, porque ningún registro está a más de unas 21 posiciones de su lugar. Inserción deja de mover un registro en cuanto encuentra su lugar y la burbuja con bandera se detiene cuando una pasada no intercambia nada. Es por eso que ambas casi no tienen trabajo en el archivo casi ordenado. En cambio, Selección siempre busca el mínimo en todo lo que queda y por eso no se “entera” de que los datos ya estaban casi en su lugar.

3. Interpretación de mis mediciones

Con 6,818 registros, Burbuja pasó de 82.7 ms a 0.40 ms y Inserción de 17.9 ms a 0.11 ms al cambiar al archivo casi ordenado, unas 150 a 200 veces más rápidas. Selección casi no cambió (105.1 ms contra 98.8 ms), y Swap Sort mejoró a la mitad o menos (85.5 ms a 21.0 ms): sus intercambios bajaron de 11,593,691 a 17,547, pero las comparaciones siguen siendo las mismas, así que no se libra del trabajo cuadrático. Shell, Merge y Quick tardaron menos de 1 ms con los dos archivos. Esto coincide con la complejidad teórica: los algoritmos con mejor caso $O(n)$ aprovechan el orden previo, y los que siempre hacen n^2 o $n \log n$ comparaciones no.

Mis predicciones coincidieron en 12 de las 14 corridas. Las dos que fallaron fueron Inserción con el archivo desordenado y Swap Sort con el casi ordenado: en ambos casos predije «Lenta» porque son $O(n^2)$, pero midieron 17.9 ms y 21.0 ms, es decir, «Media». Aprendí que la complejidad indica cómo crece el tiempo, no cuánto tarda con un tamaño concreto: con 6,818 registros, un algoritmo cuadrático todavía puede terminar en milisegundos.

Quick Sort me sorprendió: mi primera versión usaba el último registro como pivote y con el archivo casi ordenado tardó 41.2 ms, contra 1.26 ms con el desordenado. Como el último registro de cada grupo es casi el mayor, una partición se quedaba con casi todo y la otra con nada, y el algoritmo se volvía cuadrático. Al escoger el pivote con la mediana de tres bajó a 0.62 ms. También noté que los tiempos de una sola corrida varían: mi Inserción con el archivo casi ordenado midió 0.11 ms en las pruebas oficiales y 2.8 ms en una corrida mientras grababa, así que las diferencias menores a 1 ms son ruido de medición.

4. Uso de IA

Herramienta y cómo la usé. Usé Claude (Anthropic) durante todo el proyecto. Al inicio de la conversación la IA generó una versión de referencia completa; después le pedí que me explicara todo y me guiara para construir el programa yo paso a paso en VS Code, y esa es la versión que entrego: fui escribiendo cada archivo, compilando y probando cada etapa con los datos reales.

Prompts clave. (1) Necesito que me expliques la lógica de programación dentro de este programa
(2) cuáles son las diferencias claves entre algoritmos

Qué modifiqué o corregí. Escribí el código en mis propios archivos (parser.cpp, sorts.h, search.cpp, etc.) con mi encabezado de nombre y matrícula, y ejecuté cada algoritmo con los dos archivos para anotar mis propios tiempos. Al escribirlo cometí errores que arreglé con ayuda de los mensajes del compilador: el nombre de un archivo partido en dos líneas, una línea sobrante en main.cpp, la función de fechas pegada en el archivo equivocado (Parser.h en vez de parser.cpp) y una función de Quick Sort que se borró al pegar la mejora.

Casos en que la IA se equivocó o sugirió algo ineficiente. El principal es Quick Sort: la IA me propuso primero la versión más sencilla, con el último registro como pivote. Es una implementación correcta, pero ineficiente para estos datos: con el archivo casi ordenado tardó 41.2 ms (unas 33 veces más que con el desordenado). Lo detecté al medir las dos corridas, entendí que el problema era el pivote (las particiones salían de tamaño $n-1$ y 0) y lo corregí con la mediana de tres, que bajó el tiempo a 0.62 ms. El segundo caso fue que la IA armó su primera versión de referencia suponiendo una lista estándar de algoritmos, con Heap Sort y sin Swap Sort; me di cuenta porque no coincidía con mi lista y se lo corregí. Ambos me enseñaron que el código de una IA se debe verificar con mediciones y con mi propio enunciado, no darlo por bueno solo porque compila.